

SnowConvert AI Take-Home Assignment – Submission Notes

Assignment Understanding and Approach

The objective of this assignment was to migrate a SQL Server-based financial planning system to Snowflake, with primary emphasis on correctly implementing the stored procedure `usp_ProcessBudgetConsolidation`, while extending the migration to additional core workflows where time allowed.

Instead of a full one-to-one migration, I scoped the implementation to the minimum dependencies required for correctness and extended incrementally to demonstrate system completeness.

This decision allowed me to:

- maintain focus on correctness of the primary deliverable
 - control scope effectively
 - avoid unnecessary complexity early in the process
-

Migration Idea

The migration was guided by the following rule:

Preserve business logic while redesigning platform-specific implementations.

Instead of translating SQL Server code line by line, I adapted the logic to align with Snowflake's execution model.

My key transformations include:

- procedural → set-based logic
- table variables → temporary tables
- HIERARCHYID → recursive CTEs
- XML → VARIANT
- computed columns → stored values

This ensured the final solution is compatible, scalable, and easier to validate in Snowflake.

Key Design Decisions

1. Dependency-Driven Migration

The migration was performed by identifying the minimum set of dependencies required for SP1. Only the following were implemented initially:

- core tables (BudgetHeader, BudgetLineItem, CostCenter, GLAccount, FiscalPeriod)
- hierarchy logic
- rollup logic
- elimination logic

Non-essential components, such as reporting views, were deferred initially. After completing SP1, two additional procedures were implemented as scoped extensions:

- `usp_ExecuteCostAllocation` (SP2), to support rule-driven cost allocation
- `usp_ReconcileIntercompanyBalances` (SP3), to support intercompany variance detection and adjustment journal creation

This kept the migration focused while still demonstrating that the overall design could be extended beyond the minimum required procedure.

2. Hierarchy Reconstruction

SQL Server's `HIERARCHYID` functionality was replaced using a recursive Common Table Expressions (CTEs) and encapsulated in a reusable view (`vw_CostCenterHierarchy`).

The view generates:

- hierarchy level
 - hierarchical path
 - leaf node identification
-

3. Rollup Logic Redesign

The original cursor-based bottom-up rollup logic was replaced with a set-based approach using:

- an ancestor-descendant mapping
- recursive traversal
- grouped aggregation

This allows each descendant node to contribute to all its ancestor nodes, achieving correct hierarchical rollup behavior in a scalable manner.

4. Intercompany Elimination

A first-pass elimination model:

- applies to intercompany accounts
 - separates positive/negative balances
 - eliminates using minimum overlap
-

5. Idempotency and Rerun Protection

To ensure deterministic behavior, the procedure includes a rerun check:

- if a consolidated budget already exists for a given source budget, execution fails

This prevents duplicate data creation and reflects any production-grade design thinking.

6. Schema Reconstruction and Trade-offs

The schema was preserved as closely as possible while adapting unsupported SQL Server features.

Key adjustments include:

- computed columns stored explicitly (e.g., FinalAmount, IsLocked, IsBalanced)
 - XML replaced with VARIANT
 - FILESTREAM and ROWGUIDCOL removed or simplified
-

7. Cost Allocation Engine (SP2)

After completing the consolidation logic, a second stored procedure (usp_ExecuteCostAllocation) was implemented to handle cost allocation.

This procedure introduces rule-driven allocation capabilities and extends the system beyond consolidation into post-processing financial adjustments.

Key features of this implementation include:

- Rule-based allocation using AllocationRule table
 - Support for semi-structured target definitions using VARIANT (JSON)
 - Dynamic expansion of allocation targets via FLATTEN
 - Set-based allocation processing replacing cursor-based logic
 - Support for both percentage-based and calculated allocation factors
 - Traceability through AllocationSourceLineID for auditability
 - Dry-run capability to validate allocation logic before execution
 - Automatic marking of source records as allocated
-

8. Intercompany Reconciliation Engine (SP3)

A third stored procedure (usp_ReconcileIntercompanyBalances) implements intercompany reconciliation.

This procedure extends the solution into a financial control workflow by identifying intercompany imbalances, measuring variance against configurable tolerance thresholds, and optionally generating adjustment journals.

Key features of this implementation include:

- validation of source budget prior to reconciliation
- entity-level grouping of intercompany balances
- pairing of intercompany accounts using ConsolidationAccountID
- variance and variance-percentage calculation
- tolerance-based reconciliation status assignment
- optional generation of adjustment journal headers and journal lines
- summary-based output for validation and review

This procedure was implemented as a first-pass Snowflake-native redesign of the SQL Server version. XML-based reporting and entity input handling were simplified into a cleaner Snowflake approach while preserving the core reconciliation logic.

Thought Process

1. Identify minimal dependencies
2. Break logic into stages

3. Implement and validate independently
 4. Combine into final procedure
 5. Extend to allocation (SP2) and reconciliation (SP3)
-

Validation Strategy

Since production data was not provided, validation was performed using synthetic datasets.

Test scenarios included:

- multi-level cost center hierarchies
- revenue and expense aggregation
- intercompany elimination cases
- rerun behavior validation

This ensured deterministic validation and correctness across all stages.

For reconciliation validation, paired intercompany GL accounts and controlled cross-entity budget entries were used to verify pair detection, variance calculation, tolerance handling, and journal creation behavior.

Limitations and Trade-offs

Intentional simplifications include:

- Intercompany elimination is a first-pass model and does not include full partner-entity matching
- Computed columns are stored rather than dynamically derived
- SQL Server transaction semantics (e.g., savepoints) are simplified
- Validation is based on synthetic data rather than production-scale datasets
- Allocation logic currently supports percentage-based and simple factor-based methods but does not include advanced rule chaining or multi-stage dependency execution.
- Intercompany reconciliation was implemented as a first-pass version and does not include the full XML-based reporting structure from SQL Server
- Entity grouping for reconciliation was adapted to the available synthetic CostCenter test data rather than relying on delimiter-based entity codes

- Reconciliation focuses on summary-level matching and journal generation rather than full detailed line-by-line settlement logic

These trade-offs were made to prioritize correctness and maintain focus on the deliverables.

Evolution of Design Decisions

An important adjustment during development was related to schema design.

Initially, the Snowflake schema was simplified by removing several fields. However, this approach was revised to restore schema fidelity after recognizing that:

- workflow and audit fields are important for downstream logic
- removing columns reduced traceability and alignment with source behavior

This change improved the robustness and completeness of the final solution.

System Architecture

The migrated system follows a staged financial processing pipeline:

```
Budget Data
  ↓
SP1: Consolidation
  - hierarchy rollup
  - intercompany elimination
  ↓
SP2: Cost Allocation
  - rule-based distribution
  - target expansion (VARIANT)
  ↓
SP3: Intercompany Reconciliation
  - entity pairing
  - variance calculation
  - tolerance evaluation
  ↓
Adjustment Journals
  - ConsolidationJournal
  - ConsolidationJournalLine
```

This design separates transformation, allocation, and validation into independent stages, making the system modular, testable, and extensible.

Strengths of the Implementation

This submission demonstrates:

- structured, dependency-aware migration strategy
 - Snowflake-native redesign rather than direct translation
 - clear separation of logical components
 - scalable hierarchy and rollup implementation
 - production-aware considerations such as idempotency
 - validation without reliance on external data
 - explicit handling of trade-offs and limitations
 - Handling of semi-structured data using VARIANT and FLATTEN
 - Implementation of a financial control workflow with tolerance-based status logic
 - Journal creation capability for unresolved intercompany balances
-

AI Assistance Disclosure

AI tools were used as a supporting aid during development, primarily for:

- clarifying Snowflake-specific syntax and behavior
- debugging errors and validating intermediate outputs
- improving code readability and structure

All core logic, including:

- consolidation design
- hierarchy processing
- rollup implementation
- elimination logic
- idempotency handling
- allocation design
- reconciliation design

- journal generation logic

was independently designed, implemented, and validated.

Future Enhancements

Given more time, the following improvements could be implemented:

- Richer intercompany reconciliation using entity metadata and detailed matching rules
 - Automated reconciliation reporting and exception summaries
 - Enhanced allocation strategies (driver-based, statistical allocation)
 - Full intercompany elimination with partner-entity matching
 - Implementation of logging and monitoring systems
 - Addition of reconciliation and validation frameworks
 - Stronger journal balancing and offset-entry generation for production reconciliation workflows
-

Conclusion

The implementation focuses on delivering a correct, maintainable, and Snowflake-compatible financial processing pipeline, including:

- usp_ProcessBudgetConsolidation (SP1)
- usp_ExecuteCostAllocation (SP2)
- usp_ReconcileIntercompanyBalances (SP3)

Rather than attempting to migrate the entire system, the work prioritizes:

- correctness of core logic
- clarity of design
- extensibility for future enhancements

This provides a strong and practical foundation for further expansion into a complete Snowflake-based financial processing system.

Together, these procedures demonstrate consolidation, allocation, and reconciliation as three connected financial-processing workflows migrated from

SQL Server into Snowflake.

Submission of Vaishnavi Mocherla